

Name - Adarsh Kumar
PRN - 1272251304
Sem - FYMCA 'E' Sub → DS

PAGE No.	01
DATE	/ /

Sorting Techniques Assignment

1. Bubble Sort

Algorithm: BubbleSort(A, N)

Input: Array A of N elements

Output: Sorted array in ascending order

Step 1: Repeat for $i = 0$ to $N-2$

Step 2: Repeat for $j = 0$ to $N-2-i$

Step 3: if $A[j] > A[j+1]$, Then swap $A[j]$ and $A[j+1]$

Step 4: End inner loop

Step 5: End outer loop

Step 6: Exit

C++ Program

```
#include <iostream>
```

```
using namespace std;
```

```
int main()
```

```
{  
    int A[10], N = 10, temp;
```

```
    cout << "Enter 10 elements\n";
```

```
    for (int i = 0; i < N-1; i++)
```

```
    {  
        for (int j = 0; j < N-1-i; j++)
```

```
        {  
            if (A[j] > A[j+1])
```



```

while (j > 0 && A[j] > temp)
{
    A[j+1] = A[j];
    j = j-1;
}
A[j+1] = temp;
}

cout << "\n Sorted Array: \n";
for (int i = 0; i < N; i++)
    cout << A[i] << " ";

return 0;
}

```

Time Complexity

Case	Complexity
Best case (already sorted)	$O(n)$
Average case	$O(n^2)$
Worst case (Reverse order)	$O(n^2)$

Optimization

Insertion Sort automatically stops shifting once correct position is found.

Best Case occurs when array is already sorted.
Only comparisons $\rightarrow$ no shifting $\rightarrow O(n)$

No extra Optimization required in classical form.

2. Insertion Sort

Algorithm: Insertion-sort (A, N)

Input: Array A with N elements

Output: Sorted array in ascending order

- Step 1: Repeat for $i = 1$ to $N-1$
- Step 2: Repeat set $temp = A[i]$
- Step 3: Set $j = i-1$
- Step 4: while $(j \geq 0)$ and $A[j] > temp$ do
- Step 4.1: set $A[j+1] = A[j]$
- Step 4.2: Set $j = j-1$
- Step 5: Set $A[j+1] = temp$
- Step 6: end loop
- Step 7: Exit

C++ Program

```
#include <iostream>
using namespace std;
```

```
int main () {
```

```
    int A[5], N=5, temp, j;
```

```
    cout << "Enter 5 single digit numbers: ";
```

```
    for (int i=0; i<N; i++)
```

```
        cin >> A[i];
```

```
    for (int i=1; i<N; i++) {
```

```
        temp = A[i];
```

```
        j = i-1;
```


Pass 3 (i = 3)

temp = 1
j = 2
Compare A[2] = 9 > 1 → Yes
Shift 9
4 7 9 9 3
j = 1
Compare A[1] = 7 > 1 → Yes
Shift 7
4 7 7 9 3
j = 0
Compare A[0] = 4 > 1 → Yes
Shift 4
4 4 7 9 3
j = -1
Insert temp at position 0
1 4 7 9 3
Result after pass 3 :
1 4 7 9 3

Pass 4 (i = 4)

temp = 3
j = 3
Compare A[3] = 9 > 3 → Yes
Shift 9
1 4 7 9 9
j = 2
Compare A[2] = 7 > 3 → Yes
Shift 7
1 4 7 7 9
j = 1

Compare 8 and 0 $\rightarrow$ Swap

3 2 4 5 10 6 7 8 9

Result after Pass 4:

3 2 4 5 10 6 7 8 9

Pass 5

Compare 3 and 2 $\rightarrow$ Swap

2 3 4 5 10 6 7 8 9

Compare 3 and 4 $\rightarrow$ No Swap

Compare 4 and 5 $\rightarrow$ No Swap

Compare 5 and 1 $\rightarrow$ Swap

2 3 4 1 5 0 6 7 8 9

Compare 5 and 0 $\rightarrow$ Swap

2 3 4 10 5 6 7 8 9

Result after pass 5:

2 3 4 10 5 6 7 8 9

Pass 6

Compare 2 and 3 $\rightarrow$ no swap

Compare 3 and 4 $\rightarrow$ no swap

Compare 4 and 1 $\rightarrow$ Swap

2 3 1 4 0 5 6 7 8 9

Compare 4 and 0 $\rightarrow$ Swap

2 3 10 4 5 6 7 8 9

Result after Pass 6:

2 3 10 4 5 6 7 8 9

Pass 7

Compare 2 and 3 $\rightarrow$ no swap

Compare 3 and 1 $\rightarrow$ swap

2 1 3 0 4 5 6 7 8 9

Compare 3 and 0 $\rightarrow$ swap

2 1 0 3 4 5 6 7 8 9

Result after pass 7:

2 1 0 3 4 5 6 7 8 9

Pass 8:

Compare 2 and 1 $\rightarrow$ swap

1 2 0 3 4 5 6 7 8 9

Compare 2 and 0 $\rightarrow$ swap

1 0 2 3 4 5 6 7 8 9

Result after pass 8:

1 0 2 3 4 5 6 7 8 9

Pass 9:

Compare 1 and 0 $\rightarrow$ swap

0 1 2 3 4 5 6 7 8 9

Result after pass 9:

0 1 2 3 4 5 6 7 8 9

Final Sorted Array: 0 1 2 3 4 5 6 7 8 9

Tracing

Initial Array :
9 4 7 1 3

Pass 1 ($i = 1$)

temp = 4

$j = 0$

compare $A[0] = 9 > 4 \rightarrow \text{Yes}$

Shift 9 to position 1

9 9 7 1 3

$j = -1$

Insert temp at position 0

4 9 7 1 3

Result after pass 1:

4 9 7 1 3

Pass 2 ($i = 2$)

temp = 7

$j = 1$

compare $A[1] = 9 > 7 \rightarrow \text{Yes}$

Shift 9 to position 2

4 9 9 1 3

$j = 0$

compare $A[0] = 4 > 7 \rightarrow \text{No}$

Insert temp at position 1.

4 7 9 1 3

Result after pass 2:

4 7 9 1 3

Pass 3

Compare 3 and 4 $\rightarrow$ no swap.

Compare 4 and 7 $\rightarrow$ no swap.

Compare 7 and 2 $\rightarrow$ Swap.

3 4 2 7 6 5 1 0 8 9

Compare 7 and 6 $\rightarrow$ Swap

3 4 2 7 7 5 1 0 8 9

Compare 7 and 5 $\rightarrow$ Swap.

3 4 2 6 5 7 1 0 8 9

Compare 7 and 1 $\rightarrow$ Swap.

3 4 2 6 5 1 7 0 8 9

Compare 7 and 0 $\rightarrow$ Swap.

3 4 2 6 5 1 0 7 8 9

Result after pass 3:

3 4 2 6 5 1 0 7 8 9

Pass 4

Compare 3 and 4 $\rightarrow$ no swap

Compare 4 and 2 $\rightarrow$ Swap

3 2 4 6 5 1 0 7 8 9

Compare 4 and 6 $\rightarrow$ No swap.

Compare 6 and 5 $\rightarrow$ Swap.

3 2 4 5 6 1 0 7 8 9

compare $A[i] = 4 > 3 \rightarrow \text{Yes}$

Shift 4

1 4 4 7 9

$j = 0$

Compare $A[0] = 1 > 3 \rightarrow \text{No}$

Insert temp at position 1

1 3 4 7 9

Result after pass 4:

1 3 4 7 9

3. Selection Sort

Algorithm: Selection-Sort (A, N)

Input: Array A of N elements

Output: Sorted array in ascending order.

- Step 1: Repeat for $i = 0$ to $N-2$
- Step 2: Set $\text{Min} = i$
- Step 3: Repeat for $j = i+1$ to $N-1$
- Step 4: If $A[j] < A[\text{Min}]$, then set $\text{Min} = j$
- Step 5: End inner loop
- Step 6: If $\text{Min} \neq i$, then swap $A[i]$ and $A[\text{Min}]$
- Step 7: End Outer loop
- Step 8: Exit

C++ Program

```
#include <iostream>
using namespace std;
```

```
int main () {
```


3 5 9 38 43 82 10 27 12 30

Pass 4 ($i=3$)

Assume Min = 3 (38)

Smallest = 10

Swap: 3 5 9 10 43 82 38 27 12 30

Pass 5 ($i=4$)

Remaining elements:

43 82 38 27 12 30

Minimum = 12

Swap 43 and 12

3 5 9 10 12 82 38 27 43 30

Pass 6 ($i=5$)

Remaining elements

82 38 27 43 30

Min = 27

Swap 82 and 27

3 5 9 10 12 27 38 82 43 30

Pass 7 ($i=6$)

Remaining elements

38 82 43 30

Min = 30

Swap 38 and 30

3 5 9 10 12 27 30 82 43 38

Pass 8 ($i=7$)

Remaining elements: 82, 43, 38

Min = 38

Swap 82 and 38

3 5 9 10 12 27 30 38 43 82

Worst Case $O(n^2)$

No. of Comparisons $= n(n-1)/2$

No. of Swaps $\leq n-1$

Optimization

Selection sort always performs the same number of comparisons.

Only Optimization: avoid unnecessary swap when $\text{Min} = i$.

(It does not improve time complexity).

Manual Trace (10 elements)

38, 27, 43, 3, 9, 82, 10, 5, 12, 30.

Pass 1 ($i=0$)

Assume $\text{Min} = 0$ (38)

Compare with all elements $\rightarrow$ Smallest is 3.

Swap 38 and 3.

3, 27, 43, 38, 9, 82, 10, 15, 12, 30.

Pass 2 ($i=1$)

Assume $\text{Min} = 1$ (27)

Smallest in remaining = 5.

Swap 27 and 5.

3, 5, 43, 38, 9, 82, 10, 27, 12, 30.

Pass 3 ($i=2$)

Assume $\text{Min} = 2$ (43)

Smallest = 9.

Swap 43 and 9.

Pass 9 (128)

Remaining elements: 43 82

Min = 43

No swap needed

3 5 9 10 12 27 30 38 43 82

4. Quick Sort

Algorithm: Quick-Sort (A, low, High)

Input: Array A with low and High indices
Output: Sorted array

- Step 1: If low < High
- Step 2: Set loc = Partition (A, low, high)
- Step 3: Quick-Sort (A, low, loc - 1)
- Step 4: Quick-Sort (A, loc + 1, high)
- Step 5: Exit

Partition Algorithm (Classical - First Element as Pivot)

Algorithm: Partition (A, low, High)

- Step 1: Set Pivot = A[low]
- Step 2: Set Left = low + 1
- Step 3: Set Right = High
- Step 4: Repeat while Left ≤ Right
- Step 5: while (A[Left] ≤ Pivot) and Left ≤ High
Left = Left + 1
- Step 6: while (A[Right] > Pivot)
Right = Right - 1

Step 7: If $Left < Right$
 Swap $A[Left], A[Right]$.

Step 8: Repeat
 Step 9: Swap $A[low], A[Right]$

Step 10: Return Right

C++ Program

```
#include <iostream>
```

```
using namespace std;
```

```
int partition (int A[], int low, int high)
```

```
{
    int pivot = A[low];
```

```
    int left = low + 1;
```

```
    int right = high;
```

```
    int temp;
```

```
    while (left <= right) {
```

```
        while (left <= high && A[left] <= pivot)
            left++;
```

```
        while (A[right] > pivot)
            right--;
```

```
        if (left < right) {
```

```
            temp = A[left];
```

```
            A[left] = A[right];
```

```
            A[right] = temp;
```

```
        }
```

```
}
```



```

void temp = A[low];
A[low] = A[right];
A[right] = temp;

return right;
}

void quickSort(int A[], int low, int high)
{
    if (low < high) {
        int loc = partition(A, low, high);
        quickSort(A, low, loc-1);
        quickSort(A, loc+1, high);
    }
}

int main() {
    int A[10], N = 10;
    cout << "Enter 10 elements: \n";

    for (int i = 0; i < N; i++)
        cin >> A[i];

    quickSort(A, 0, N-1);

    cout << "In sorted array: \n";
    for (int i = 0; i < N; i++)
        cout << A[i] << " ";

    return 0;
}

```


Right half:

83 10 | 5 12 30

82 | 10

5 | 12 30

12 | 30

Step 2: Merge phase

Merge 38 & 27 $\Rightarrow$
27 38

Merge 3 & 9 $\Rightarrow$
3 9

Merge 43 & (3, 9) $\Rightarrow$
3 9 27 38 43

Right Side:

Merge 82 & 10 $\Rightarrow$
10 82

Merge 12 & 30 $\Rightarrow$
12 30

Merge 5 & (12, 30) $\Rightarrow$
5 12 30

Merge (10 82) & (5, 12 30) $\Rightarrow$
5 10 12 30 82

Final Merge

Merge

3 9 27 38 43
and

5 10 12 30 82

Final Result

3 5 9 10 12 27 30 38 43 82

Time Complexity

Case
Best Case
Average Case
Worst Case

Complexity

$O(n \log n)$
 $O(n \log n)$
 $O(n^2)$

Optimization

1. Choose random pivot
2. Choose middle element
3. Use median-of-three method.

Tracing

Pivot = 0 $\rightarrow$ placed at correct position
left empty | right unsorted
Repeat $\rightarrow$ final sorted array

5) Merge Sort

Algorithm: Merge-Sort(A, low, high)

Input : Array A with low and high indices
Output : Sorted array

- Step 1: If low < high
- Step 2: Set mid = (low + high) / 2
- Step 3: Merge-Sort(A, low, mid)
- Step 4: Merge-Sort(A, mid+1, high)
- Step 5: Merge(A, low, mid, high)
- Step 6: Exit


```

if (left < n && A[left] > A[largest])
    largest = left;
if (right < n && A[right] > A[largest])
    largest = right;
if (largest != i)
{
    temp = A[i];
    A[i] = A[largest];
    A[largest] = temp;
    heapify(A, N, largest);
}
}

```

```

void heapsort(int A[], int n)
{
    for (int i = n/2 - 1; i >= 0; i--)
        heapify(A, n, i);
    for (int i = n-1; i > 0; i--)
    {
        int temp = A[0];
        A[0] = A[i];
        A[i] = temp;
        heapify(A, i, 0);
    }
}

```

```

int main ()
{
    int A[10], N = 10;
    cout << "Enter 10 elements: \n";
    for (int i = 0; i < N; i++)
        cin >> A[i];
}

```


Heapify :

38 30 10 12 27 9 3 5 43 82

Swap 38 and 5

5 30 10 12 27 9 3 38 43 82

Heapify :

30 27 10 12 5 9 3 38 43 82

7)

Radix Sort

Algorithm : Radix-Sort (A, N)

Input : Array A of N integers

Output : Sorted array

Step 1: Find the maximum number in A

Step 2: Set Exp = 1

Step 3: While (Max(Exp) > 0)

Step 4: Perform Counting Sort (A, N, Exp)

Step 5: Set Exp = Exp * 10

Step 6: End While

Step 7: Exit

C++ Program

```
#include <iostream>
```

```
using namespace std;
```

```
int getMax(int A[], int n)
```

```
{
    int max = A[0];
```



```

int main ()
{
    int A[10], N = 10;
    cout << "Enter 10 elements: \n";
    for (int i = 0; i < N; i++)
        cin >> A[i];

    radixSort (A, N);

    cout << "\n Sorted array: \n";
    for (int i = 0; i < N; i++)
        cout << A[i] << " ";

    return 0;
}

```

Time complexity

Let:

n = Number of elements

d = no. of digits

<u>Case</u>	<u>Complexity</u>
Best	$O(nk)$
Average	$O(nk)$
Worst	$O(nk)$

Manual Trace

Eg: 38 27 43 3 9 82 10 5 12 30

Step 1: Sort by Units Digit (exp = 1)


```

for (int i = 1; i < n; i++)
    if (A[i] > max)
        max = A[i];
return max;
}

void countingSort (int A[], int n, int exp)
{
    int output [10];
    int count [10] = {0};

    for (int i = 0; i < n; i++)
        count [(A[i] / exp) % 10]++;

    for (int i = 0; i < 10; i++)
        count [i] += count [i-1];

    for (int i = n-1; i >= 0; i--)
    {
        int digit = (A[i] / exp) % 10;
        output [count [digit] - 1] = A[i];
        count [digit]--;
    }

    for (int i = 0; i < n; i++)
        A[i] = output [i];
}

void radixSort (int A[], int n)
{
    int max = getMax(A, n);

    for (int exp = 1; max / exp > 0; exp *= 10)
        countingSort (A, n, exp);
}

```


Units digits :

38 $\rightarrow$ 8
 27 $\rightarrow$ 7
 43 $\rightarrow$ 3
 3 $\rightarrow$ 3
 9 $\rightarrow$ 9
 82 $\rightarrow$ 2
 10 $\rightarrow$ 0
 5 $\rightarrow$ 5
 12 $\rightarrow$ 2
 30 $\rightarrow$ 0

After counting Sort (by units) :

10 30 82 12 43 3 5 27 38 9

Step 2: Sort by Tens Digit (exp = 10)

Tens digits :

10 $\rightarrow$ 1
 30 $\rightarrow$ 3
 82 $\rightarrow$ 8
 12 $\rightarrow$ 1
 43 $\rightarrow$ 4
 3 $\rightarrow$ 0
 5 $\rightarrow$ 0
 27 $\rightarrow$ 2
 38 $\rightarrow$ 3
 9 $\rightarrow$ 0

After counting Sort (by tens):

3 5 9 10 12 27 30 38 43 82


```

heapSort (A, N);
cout << "In sorted array: \n";
for (int i = 0; i < N; i++)
    cout << A[i] << " ";
return 0;
}

```

Time Complexity

<u>Case</u>	<u>Complexity</u>
Best Case	$O(n \log n)$
Average Case	$O(n \log n)$
Worst Case	$O(n \log n)$

Manual Trace

Eg: 38, 27, 43, 3, 9, 82, 10, 5, 12, 30.

Step 1: Build Max Heap.
 After heap construction:
 82 30 43 12 27 38 10 5 3 9
 (Max element 82 at root)

Step 2: Delete root Repeatedly
 Swap 82 and 9:
 9 30 43 12 27 38 10 5 3 82
 Heapify:
 43 30 38 12 27 9 10 5 3 82

Swap 43 and 3:
 3 30 38 12 27 9 10 5 43 82

6) Heap Sort

Heap sort works in two phases:

- Build a Max Heap
- Repeatedly delete the root (largest element)

In Max Heap:

Parent $\geq$ children

Algorithm: Heap-Sort (A, N)

Input: Array A of N elements

Output: Sorted array in ascending order

Step 1: Build-Max-Heap (A, N)

Step 2: Repeat for $i = N-1$ down to 1

Step 3: Swap $A[0]$ and $A[i]$

Step 4: Call Heapify (A, 0, $i-1$)

Step 5: End loop

Step 6: Exit

Algo C++ Program

```
#include <iostream>
```

```
using namespace std;
```

```
void heapify (int A[], int n, int i)
```

```
{
```

```
    int largest = i;
```

```
    int left = 2 * i + 1;
```

```
    int right = 2 * i + 2;
```

```
    int temp;
```


22
DATE / /

```
    B[k++] = A[j++];  
}
```

```
while (i <= mid)  
    B[k++] = A[i++];
```

```
while (j <= high)  
    B[k++] = A[j++];
```

```
for (int x = low; x <= high; x++)  
    A[x] = B[x];  
}
```

```
void mergesort (int A[], int low, int high) {  
    if (low < high) {  
        int mid = (low + high) / 2;  
        mergesort (A, low, mid);  
        mergesort (A, mid + 1, high);  
        merge (A, low, mid, high);  
    }  
}
```

```
int main () {  
    int A[10], N = 10;
```

```
    cout << "Enter 10 elements: \n";  
    for (int i = 0; i < N; i++)  
        cin >> A[i];
```

```
    mergesort (A, 0, N - 1);  
    cout << "Sorted array: \n";  
    for (int i = 0; i < N; i++)  
        cout << A[i] << " ";  
    return 0;
```

```
}
```


Time Complexity

<u>Case</u>	<u>Complexity</u>
Best Case	$O(n \log n)$
Average Case	$O(n \log n)$
Worst Case	$O(n \log n)$

Number of levels $= \log n$

Work per level $= n$

Total $= n \log n$

Optimization

1. Use insertion sort for very small subarrays.
2. Avoid copying if already sorted.
3. Use single auxiliary array globally.

Manual Trace

Eg: 38, 27, 43, 3, 9, 82, 10, 5, 12, 30

Step 1: Divide Phase

split into halves:

38 27 43 3 9 | 82 10 5 12 30

Left half:

38 | 27 | 43 3 9

Further split:

38 | 27

43 | 3 9

3 | 9

Algorithm : Merge (A, low, mid, high)

- Step 1: Create temporary array B
- Step 2: Set $i = \text{low}$
- Step 3: Set $j = \text{mid} + 1$
- Step 4: Set $k = \text{low}$
- Step 5: while $i < \text{mid}$ and $j \leq \text{high}$
 - if $A[i] \leq A[j]$
 - Set $B[k] = A[i]$, $i = i + 1$
 - else
 - Set $B[k] = A[j]$, $j = j + 1$
 - $k = k + 1$
- Step 6: Copy remaining elements of left Subarray
- Step 7: Copy remaining elements of right subarray
- Step 8: Copy B back to A
- Step 9: Exit

C++ Program

#include <iostream>

using namespace std;

```
void merge (int A[], int low, int mid, int high) {
    int B[10];
    int i = low;
    int j = mid + 1;
    int k = low
```

```
    while (i <= mid && j <= high)
    {
```

```
        if (A[i] <= A[j])
            B[k++] = A[i++];
        else
```


Compare 9 and 0 $\rightarrow$ swap

3 7 4 8 2 6 5 1 0 9

Result after pass 1:

3 7 4 8 2 6 5 1 0 9

Pass 2

Compare 3 and 7 $\rightarrow$ No Swap

3 7 4 8 2 6 5 1 0 9

Compare 7 and 4 $\rightarrow$ swap

3 4 7 8 2 6 5 1 0 9

Compare 7 and 8 $\rightarrow$ no swap

Compare 8 and 2 $\rightarrow$ swap

3 4 7 2 8 6 5 1 0 9

Compare 8 and 6 $\rightarrow$ swap

3 4 7 2 6 8 5 1 0 9

Compare 8 and 5 $\rightarrow$ swap

3 4 7 2 6 5 8 1 0 9

Compare 8 and 1 $\rightarrow$ swap

3 4 7 2 6 5 1 8 0 9

Compare 8 and 0 $\rightarrow$ swap

3 4 7 2 6 5 1 0 8 9

Result after Pass 2:

3 4 7 2 6 5 1 0 8 9


```
int A[10], N = 10, temp;
```

```
cout << "Enter 10 elements: \n";
```

```
for (int i = 0; i < N; i++)
```

```
    cin >> A[i];
```

```
for (int i = 0; i < N-1; i++) {
```

```
    int min = i;
```

```
    for (int j = i+1; j < N; j++)
```

```
    {
```

```
        if (A[j] < A[min]) {
```

```
            min = j;
```

```
        }
```

```
        if (min != i)
```

```
        {
```

```
            temp = A[i];
```

```
            A[i] = A[min];
```

```
            A[min] = temp;
```

```
        }
```

```
    }
```

```
    cout << "In sorted array: \n";
```

```
    for (int i = 0; i < N; i++)
```

```
        cout << A[i] << " ";
```

```
    return 0;
```

```
}
```

Time complexity

Case

Complexity

Best Case

$O(n^2)$

Average Case

$O(n^2)$

Bubble Sort Full Tracing

Initial Array

8 3 7 4 9 2 6 5 10

$N = 10$

Total passes = $N - 1 = 9$ passes

Pass 1

Compare 8 and 3 $\rightarrow$ Swap

3 8 7 4 9 2 6 5 10

Compare 8 and 7 $\rightarrow$ Swap

3 7 8 4 9 2 6 5 10

Compare 8 and 4 $\rightarrow$ Swap

3 7 4 8 9 2 6 5 10

Compare 8 and 9 $\rightarrow$ no swap

3 7 4 8 9 2 6 5 10

Compare 9 and 2 $\rightarrow$ Swap

3 7 4 8 2 9 6 5 10

Compare 9 and 6 $\rightarrow$ Swap

3 7 4 8 2 6 9 5 10

Compare 9 and 5 $\rightarrow$ Swap

3 7 4 8 2 6 5 9 10

Compare 9 and 1 $\rightarrow$ Swap

3 7 4 8 2 6 5 1 9 10


```

temp = A[j];
A[j] = A[j+1];
A[j+1] = temp;
}
}

cout << "\n Sorted array : ";
for (int i = 0; i < N; i++)
    cout << A[i] << " ";

return 0;
}

```

Time Complexity

<u>Case</u>	<u>Complexity</u>
Best Case	$O(n^2)$
Average Case	$O(n^2)$
Worst Case	$O(n^2)$

No. of Comparisons $= n(n-1)/2$

Optimization

Optimization using flag to stop early if sorted.

Algorithm improvement Step:

Add flag variable:
 If no swap in a pass $\rightarrow$ stop
 Improved Best Case $\rightarrow O(n)$

Optimized code

#include <iostream>

using namespace std;

int main () {

 int A[10], N=10, temp;
 bool swapped;

cout << "Enter 10 elements: \n";

 for (int i = 0; i < N; i++)
 cin >> A[i];

for (int i = 0; i < N-1; i++) {

swapped = false;

 for (int j = 0; j < N-1-i; j++)
 {

 if (A[j] > A[j+1])
 {

temp = A[j];

A[j] = A[j+1];

A[j+1] = temp;

swapped = true;

}

}

 if (swapped == false)
 break;

}

cout << "\n sorted array: \n";

 for (int i = 0; i < N; i++)
 cout << A[i] << " ";

return 0;

}